2022 IEEE International Conference on Image Processing (ICIP) | 978-1-6654-9620-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICIP46576.2022.9897283

ANOMALIB: A DEEP LEARNING LIBRARY FOR ANOMALY DETECTION

Samet Akcay, Dick Ameln, Ashwin Vaidya, Barath Lakshmanan, Nilesh Ahuja, Utku Genc

Intel

{ name.surname }@intel.com

ABSTRACT

This paper introduces *anomalib*[1], a novel library for unsupervised anomaly detection and localization. With reproducibility and modularity in mind, this open-source library provides algorithms from the literature and a set of tools to design custom anomaly detection algorithms via a plug-and-play approach. *Anomalib* comprises state-of-the-art anomaly detection algorithms that achieve top performance on the benchmarks and that can be used off-the-shelf. In addition, the library provides components to design custom algorithms that could be tailored towards specific needs. Additional tools, including experiment trackers, visualizers, and hyperparameter optimizers, make it simple to design and implement anomaly detection models. The library also supports OpenVINO model-optimization and quantization for real-time deployment. Overall, *anomalib* is an extensive library for the design, implementation, and deployment of unsupervised anomaly detection models from data to the edge.

***Index Terms*—** Unsupervised Anomaly detection, localization

1. INTRODUCTION

Anomaly detection is a growing research area in machine learning literature, where the goal is to distinguish between normal samples anomalous samples in a dataset. Supervised approaches are generally ineffective for this type of problem due to a lack of sufficient representative samples in the anomalous class. To address this problem, anomaly detection algorithms solely rely on normal samples during the training stage, and identify anomalous samples by comparing against the learned distribution of normal data. This unsupervised nature of anomaly detection makes it well suited for real-world applications such as industrial, medical and security, where a clearly defined anomalous class is often lacking.

The increasing number of publications and available techniques in the anomaly detection field (Figure 1) call for the need for a unified library for benchmarking algorithms. Where supervised tasks have seen various such libraries [1] [2] emerge over the past years, the unsupervised anomaly detection domain lacks similar efforts to date.

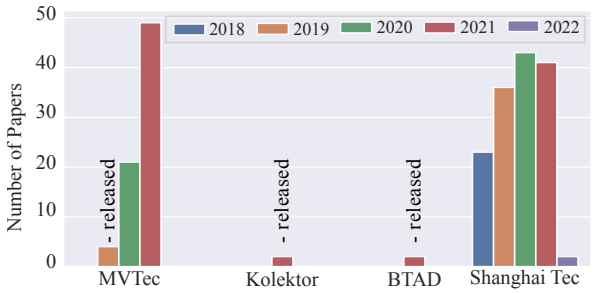

Fig. 1: Number of anomaly detection datasets and papers recently released and published in the literature.[2]

Existing anomaly detection libraries focus on single algorithms only, lack performance optimizations, or do not include deep learning techniques [3]. This makes it challenging to utilize these implementations for out-of-the-box comparison of the most recent algorithms on a given dataset. To address these issues, we introduce *anomalib*, a new library that aims to provide a complete collection of recent deep learning-based anomaly detection techniques and tools.

By collecting different anomaly detection algorithms and model components in a single library, *anomalib* provides the following advantages:

- state-of-the-art anomaly detection models to run benchmark on public and custom datasets.
- Modular anomaly model components to design new algorithms via plug-and-play.
- CLI-based, configurable entrypoints for training, testing, inference, hyperparameter optimization and benchmarking.
- Inference interfaces, compatible with number of exportable model formats that facilitate real-time local or edge deployment.

Overall, *anomalib* is a unified library that provides a set of components and tools that could be used in anomaly detection research and production.

[1] https://github.com/openvinotoolkit/anomalib

[2] Paper statistics are taken from https://paperswithcode.com

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on February 02,2026 at 12:56:52 UTC from IEEE Xplore. Restrictions apply.

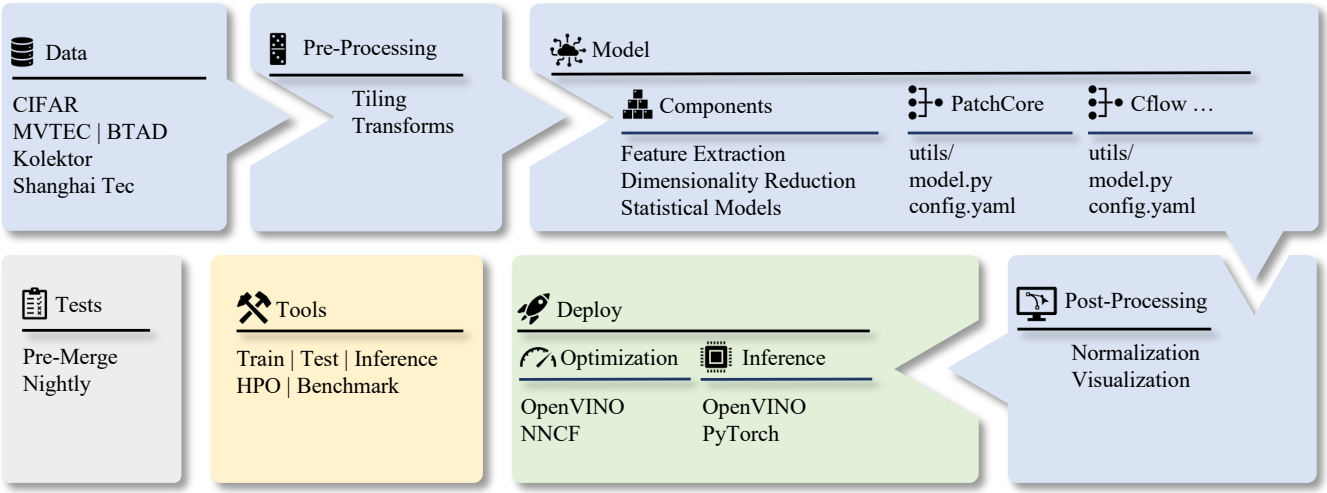

Fig. 2: Architecture of the *Anomalib* library from data to deployment.

2. DESIGN PRINCIPLES

Anomalib follows four design principles, each of which is explained below.

Reproducibility. One of the main goals of the *anomalib* library is to compare the performance of different state-of-the-art anomaly detection algorithms on public and custom benchmark datasets. To ensure a valid comparison, the algorithm implementations in *anomalib* aim to reproduce the results reported in the original publications.

Extensibility. Given the fast pace of progress in the anomaly detection field, it is crucial that new algorithms can be added to the library with minimal effort. To this end, *anomalib* provides several interfaces that developers can implement to make their models compatible with the training and inference entrypoints of the library.

Modularity. The library contains several ready-to-use components that can serve as building blocks when creating new algorithms. Developers and researchers can re-use these components in a plug-and-play fashion to further reduce implementation efforts and quickly prototype new ideas.

Real-Time Performance. A key objective of *anomalib* is to reduce the effort of performing inference with trained models. The library therefore provides interfaces to deploy models in real-time using either GPU or CPU via PyTorch [4] and OpenVINO [5] deployment options, respectively.

By following the design principles outlined above, *anomalib* aims to cover the full machine learning model lifecycle from data to deployment, where the results of existing algorithms can be reproduced, new datasets and algorithms can be added, and models can be deployed in real-time.

3. ANOMALIB

This section categorizes the library on a component level,[3] each of which is a fundamental step in the *data-to-deployment* workflow shown in Figure 2 .

3.1. Data

The library provides dataset adapters for a growing number of public benchmark datasets both from image and video domains that are widely used in the literature.

Image. *Anomalib* supports CIFAR-10 [6] for fast prototyping, and MVTec [7], BTAD [8] and Kolektor [9] for real-world defect detection applications.

Video. The library supports video datasets such as ShanghaiTec [10]. Currently, video datasets are only supported on a frame-level basis since the existing *anomalib* models are optimized for image domain. We plan to support video anomaly detection models in future releases to address this.

Custom. In addition to the aforementioned public datasets, *anomalib* provides a dataset interface for the users to implement custom datasets, on which new and existing *anomalib* models can be trained.

3.2. Pre-Processing

Pre-processing consists of applying transformations to the input images before training, and optionally dividing the images into (non-)overlapping tiles.

Transforms. *Anomalib* utilizes the *albumentations* [11] library for image transformations, which has the advantage of managing the transformation of the ground truth pixel maps together with the input images. In addition to its extensible Python API, *albumentations* allows reading transformation

[3]For more details, refer to the documentation.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on February 02,2026 at 12:56:52 UTC from IEEE Xplore. Restrictions apply.

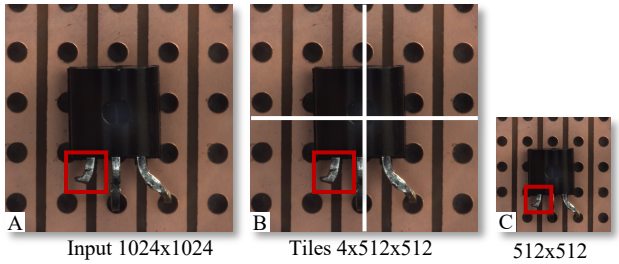

Fig. 3: Tiling vs. resizing the image to the target input size.

settings from a config file, which is a useful feature for experimentation and HPO.

Tiling. Due to the high image resolution in many real-world datasets, it is generally required to resize the input images before presenting them to the model. As a side effect, small anomalous regions within the images may lose detail, making it more challenging for the model to detect these regions. Tiling the input images alleviates the issue since the size of the anomalous regions remains consistent (Figure 3).

3.3. Model

Anomalib contains a selection of state-of-the-art anomaly detection/localization algorithms as well as a set of modular components that serve as building blocks to compose custom algorithms.

Algorithms. The library is updated periodically with the latest state-of-the-art anomaly detection models. Currently available models could be categorized into density estimation [12, 13, 14, 15], reconstruction [16] and knowledge distillation models [17].

Components. The model components comprise several ready-to-use modules that implement commonly used operations. Similar to Scikit Learn [18], the model components are categorized with respect to their role in anomaly detection models (e.g. feature extraction, dimensionality reduction, statistical modeling). All model components are implemented in PyTorch, which allows running all operations on the GPU and exporting the models to ONNX and OpenVINO.

Using the model components to implement a custom anomaly detection algorithm is straightforward. For instance, consider an anomaly model that initially extracts features via CNN and performs a dimensionality reduction via Coreset Sampling [19], similar to PatchCore [15]. One could import the components as follows:

```
from anomalib.models.components import (
    FeatureExtractor, KCenterGreedy
)
```

3.4. Post-Processing

Normalization. The range of image-level or pixel-level anomaly scores predicted by the models in *anomalib* during inference may vary depending on the model and dataset. To convert the raw anomaly scores into a standardized format, *anomalib* normalizes the predicted anomaly scores to the [0,1] range. By default, *anomalib* uses min-max normalization with respect to the values observed during validation (Figure 4), but the normalization method can be configured or disabled entirely.

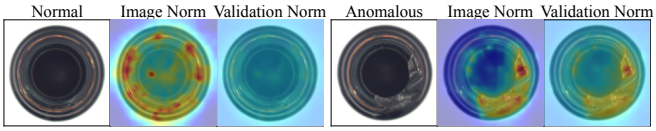

Fig. 4: Normalizing with respect to the validation set results in better visualizations than normalizing with respect to individual images.

Thresholding. To help the user choose an anomaly score threshold for their trained models, *anomalib* provides an adaptive thresholding mechanism, which optimizes the value of the threshold based on the F1 score during validation. Alternatively, the user can specify a manual threshold. This is useful in cases where insufficient representative validation data is available.

Visualization. During validation and testing, *anomalib* can be configured to show and save visualizations of the predicted anomaly heatmaps and segmentation masks (Figure 5).

3.5. Deploy

Optimization. To achieve faster inference and throughput via quantization and optimization, the library utilizes OpenVINO [5] and Neural Network Compression Framework (NNCF) [20].

Inference. Trained models can be deployed by relying solely on the library's inference utilities, which allow users to visualize the inference results in a window or save predicted anomaly scores to the file system.

3.6. Utilities

Anomalib uses a number of utilities and helper modules to facilitate a complete training and inference pipeline.

Callbacks. The library utilizes PyTorch Lightning's callback class [21] for non-model-specific operations such as normalization, OpenVINO/NNCF compression, timer and visualization to reduce boilerplate code in the model implementations.

Metrics. Commonly reported performance metrics such as AUROC, F1 and PRO are reported after each training run. The metrics are implemented using TorchMetrics [21], which allows running the computations on GPU.

Logging. The library supports multiple logging targets such as TensorBoard [22] and wandb [23] to track experiments. The library provides an interface so that the implementation details of the loggers remain abstracted from the user.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on February 02,2026 at 12:56:52 UTC from IEEE Xplore. Restrictions apply.

	Carpet	Grid	Leather	Tile	Wood	Bottle	Cable	Capsule	Hazelnut	Metalnut	Pill	Screw	Toothbrush	Transistor	Zipper	Mean
DFM [12]	0.847	0.496	0.930	0.981	0.957	0.997	0.927	0.876	0.985	0.946	0.816	0.761	**0.964**	0.936	0.945	0.891
STFPM [17]	0.921	**0.983**	0.921	0.961	**0.994**	0.998	0.946	0.717	0.996	0.984	0.492	0.550	0.638	0.852	0.837	0.853
PADIM [13]	0.945	0.857	0.982	0.950	0.976	0.994	0.843	0.901	0.750	0.961	0.863	0.759	0.889	0.920	0.780	0.891
CFLOW [14]	**0.981**	0.963	**1.000**	0.997	0.991	**1.000**	0.969	0.970	**1.000**	0.991	**0.981**	0.767	0.911	0.950	**0.986**	0.964
PatchCore [15]	0.979	0.962	**1.000**	**1.000**	0.992	**1.000**	**0.993**	**0.976**	**1.000**	**0.996**	0.934	**0.947**	0.947	**1.000**	0.982	**0.981**

Table 1: Image Level AUROC Scores for MVTec [7] dataset categories.

	Carpet	Grid	Leather	Tile	Wood	Bottle	Cable	Capsule	Hazelnut	Metalnut	Pill	Screw	Toothbrush	Transistor	Zipper	Mean
STFPM [17]	0.973	**0.987**	0.976	0.964	**0.958**	0.968	0.940	0.957	0.979	0.965	0.867	0.938	0.148	0.759	**0.983**	0.891
PADIM [13]	0.984	0.918	0.994	0.934	0.947	0.983	0.965	0.984	0.978	0.970	0.957	0.978	**0.988**	0.968	0.979	0.968
CFLOW [14]	0.985	0.969	**0.995**	**0.966**	0.926	0.984	0.965	**0.988**	**0.988**	0.982	**0.987**	0.982	0.985	0.930	0.980	0.974
PatchCore [15]	**0.988**	0.969	0.991	0.961	0.935	**0.985**	**0.988**	**0.988**	0.987	**0.989**	0.981	**0.989**	**0.988**	**0.982**	**0.983**	**0.980**

Table 2: Pixel Level AUROC Scores for MVTec [7] dataset categories.

4. LIBRARY TOOLS

4.1. Command Line Interface (CLI)

Anomalib provides several ready-to-use scripts to train and export models, run inference on trained PyTorch or OpenVINO models, and run Benchmarking and Hyperparameter Optimization experiments.

Training, Testing and Inference. *Anomalib* provides a set of python scripts for basic training (`train.py`), testing (`test.py`) and inference (`inference.py`) functionality. Each script has a set of command line arguments that can be used to configure the dataset, model and hyperparameter settings. Visualization results and model files will be saved to a file system location specified by the user. The inference entrypoint script supports both PyTorch (`.ckpt`) and OpenVINO (`.bin`, `.xml`) models, depending on the extension of specified model file.

Hyperparameter Optimization. The library provides hyperparameter optimization (HPO) support using a Weights & Biases (wandb) [23] plugin. Settings of the HPO sweep such as included parameters, monitored performance metric and number of experiments can be configured from the provided *sweep.yaml* file.

Benchmarking. The library contains a suite of benchmarking scripts for collecting statistical and computational metrics across multiple models or datasets. The entrypoint is a Python script (`benchmark.py`), which is used to perform a grid search and log the results to TensorBoard, wandb, or a local `.csv` file.

4.2. Python API

In addition to the CLI entrypoints, it is also possible to use the Python API for more flexible use of the library or when designing custom algorithms. The following code block demonstrates how a PatchCore [15] model could be trained and test on MVTec bottle [7] category.

```
from anomalib.data import Mvtec
from anomalib.models import Patchcore
```

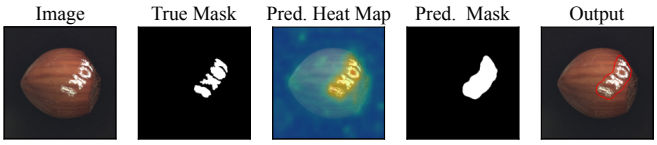

Fig. 5: An exemplary output produced by the *anomalib* visualization tools.

```
from pytorch_lightning import Trainer

datamodule = Mvtec(category="bottle", ...)
model = Patchcore(backbone="resnet18", ...)
trainer = Trainer(...)
trainer.fit(datamodule, model)
trainer.test(datamodule, model)
```

5. BENCHMARKS

Tables 1 and 2 demonstrate image-level and pixel-level AUROC scores benchmarked and averaged on MVTec [7] categories [4] for the different public models currently implemented in *anomalib* . The results were obtained by using *anomalib* 's benchmarking tool and illustrate how *anomalib* can be used to perform a comparative study between different models and dataset categories.

6. CONCLUSIONS

We introduce *anomalib*, a comprehensive library for training, benchmarking, deploying and developing deep-learning based anomaly detection models. The library provides a set of tools that allow quick and reproducible comparison of different anomaly detection models on any dataset, as illustrated by the benchmarking experiments described in this paper. We release it as an open-source package with the aim of constantly updating with the latest state-of-the-art techniques in the field, and welcome the community contribution. In future work we plan to extend *anomalib* to other domains such as audio, video, and 3-dimensional data.

[4] Refer to anomalib for more benchmark results

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on February 02,2026 at 12:56:52 UTC from IEEE Xplore. Restrictions apply.

7. REFERENCES

[1] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin, "MMDetection: Open MMLab Detection Toolbox and Benchmark," *ArXiv*, 6 2019. 1

[2] Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo, and Ross Girshick, "Detectron2," https://github.com/facebookresearch/detectron2, 2019. 1

[3] Yue Zhao, Zain Nasrullah, and Zheng Li, "PyOD: A Python Toolbox for Scalable Outlier Detection," *Journal of Machine Learning Research*, vol. 20, no. 96, pp. 1–7, 1 2019. 1

[4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *NeurIPS*, 12 2019. 2

[5] Alexander Demidovskij, Yury Gorbachev, Mikhail Fedorov, Iliya Slavutin, Artyom Tugarev, Marat Fatekhov, and Yaroslav Tarkan, "OpenVINO deep learning workbench: Comprehensive analysis and tuning of neural networks inference," *ICCVW*, pp. 783–787, 10 2019. 2, 3

[6] Alex Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., 2009. 2

[7] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger, "MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," in *2019 IEEE/CVF CVPR*. 6 2019, pp. 9584–9592, IEEE. 2, 4

[8] Pankaj Mishra, Riccardo Verk, Daniele Fornasier, Claudio Piciarelli, and Gian Luca Foresti, "VT-ADL: A Vision Transformer Network for Image Anomaly Detection and Localization," in *IEEE International Symposium on Industrial Electronics*, 4 2021. 2

[9] Domen Tabernik, Samo Šela, Jure Skvarč, and Danijel Skočaj, "Segmentation-based deep-learning approach for surface-defect detection," *Journal of Intelligent Manufacturing*, vol. 31, no. 3, pp. 759–776, 3 2020. 2

[10] Weixin Luo, Wen Liu, and Shenghua Gao, "A Revisit of Sparse Coding Based Anomaly Detection in Stacked RNN Framework," in *ICCV*. 10 2017, pp. 341–349, IEEE. 2

[11] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin, "Albumentations: Fast and Flexible Image Augmentations," *MDPI*, vol. 11, no. 2, pp. 125, 2 2020. 2

[12] Nilesh A. Ahuja, Ibrahima Ndiour, Trushant Kalyanpur, and Omesh Tickoo, "Probabilistic Modeling of Deep Features for Out-of-Distribution and Adversarial Detection," *ArXiv*, 9 2019. 3, 4

[13] Thomas Defard, Alexander Setkov, Angelique Loesch, and Romaric Audigier, "PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization," in *ICPR International Workshops and Challenges*, Cham, 2021, pp. 475–489, Springer International Publishing. 3, 4

[14] Denis Gudovskiy, Shun Ishizaka, and Kazuki Kozuka, "CFLOW-AD: Real-Time Unsupervised Anomaly Detection with Localization via Conditional Normalizing Flows," in *WACV*. 7 2021, IEEE. 3, 4

[15] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler, "Towards Total Recall in Industrial Anomaly Detection," *ArXiv*, 6 2021. 3, 4

[16] Samet Akcay, Amir Atapour-Abarghouei, and Toby Breckon, "GANomaly: Semi-supervised Anomaly Detection via Adversarial Training," in *ACCV 2018*, Cham, 2019, pp. 622–637, Springer International Publishing. 3

[17] Guodong Wang, Shumin Han, Errui Ding, and Di Huang, "Student-Teacher Feature Pyramid Matching for Unsupervised Anomaly Detection," in *BMVC*, 2021. 3, 4

[18] Fabian Pedregosa, Vincent Michel, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Jake Vanderplas, David Cournapeau, Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Bertrand Thirion, Olivier Grisel, Vincent Dubourg, Alexandre Passos, and Matthieu Brucher, "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, no. 85, pp. 2825–2830, 2011. 3

[19] Ozan Sener and Silvio Savarese, "Active Learning for Convolutional Neural Networks: A Core-Set Approach," in *ICLR*, 8 2018. 3

[20] Alexander Kozlov, Ivan Lazarevich, Vasily Shamporov, Nikolay Lyalyushkin, and Yury Gorbachev, "Neural Network Compression Framework for fast model inference," *ArXiv*, 2 2020. 3

[21] William Falcon, Jirka Borovec, Adrian Wälchli, Nic Eggert, Justus Schock, Jeremy Jordan, Nicki Skafte, Vadim Bereznyuk, Ethan Harris, Tullie Murrell, Peter Yu, Sebastian Præsius, Travis Addair, Jacob Zhong, Dmitry Lipin, So Uchida, Shreyas Bapat, Hendrik Schröter, Boris Dayma, Alexey Karnachev, Akshay Kulkarni, Shunta Komatsu, Hadrien Mary, Donal Byrne, Cristobal Eyzaguirre, and Anton Bakhtin, "PyTorch Lightning," 5 2020. 3

[22] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng, "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," Tech. Rep., 2015. 3

[23] Lukas Biewald, "Experiment Tracking with Weights and Biases," 2020. 3, 4

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on February 02,2026 at 12:56:52 UTC from IEEE Xplore. Restrictions apply.